integration test: you're assuming that a test database has been configured to return the right status for flight FH-101.

If you wanted to have more direct control over the flight status, you could easily replace the `flightService` object ❷ with your own stub:

```
flightService = {
    getStatus: function(flightNumber) {
        switch(flightNumber)
        {
            case 'FH-101': return 'ontime';
            case 'FH-102': return 'delayed';
        }
    }
};
```

Both would be valid strategies: the first focuses on ensuring that all the components are wired together correctly, whereas the second is more interested in exploring different scenarios.

Testing against the controller layer is certainly a lot faster than web testing, but it does have some limitations. For more involved scenarios, the code can be complicated to set up and hard to maintain compared to an equivalent web test. When you check the routing instructions or error messages produced by a controller component, you have no way of knowing if this routing goes to the correct web page or whether the error messages are rendered correctly. You may need to complement your controller-level tests with a few UI-based tests that illustrate these more visual aspects.

This approach is also closer to unit or integration testing than the end-to-end web tests, which may inspire less confidence from nontechnical team members. Compared to web tests, where they can see visual feedback about the application's behavior, stakeholders will have a much harder time understanding (and therefore trusting) the results of controller-layer tests. When deciding how to implement your acceptance criteria, there's often a balance to be struck between confidence in the tests and speed of execution.

9.3.2 Testing business logic directly

Some application features and some kinds of business logic are relatively independent of the UI. The credit card eligibility rule we discussed earlier is a good example of this:

```
Scenario Outline: Credit card eligibility based on income and status
Given Joe is a <status> Frequent Flyer earning <income>
When Joe applies for a Flying High Credit Card
Then his application should be <outcome>
Example:
status | income | outcome    | notes
gold   | 80000  | automatic  | Automatically approved for >= $80000
gold   | 79999  | manual     |
gold   | 49999  | declined   |
silver | 100000 | automatic  | Automatically approved for >= $100000
silver | 99999  | manual     |
bronze | 110000 | automatic  | Automatically approved for >= $110000
bronze | 109999 | manual     |
```